Java Mastery Notes

JavaMastery Notes — From OOP foundations to modern Java, everything needed before Spring Boot.
5 Milestones Complete · OOP · Collections · Exceptions · Streams · Maven · JSON · *Beginner → Industry Ready*

Module 00 — Java Theory (One-Page Foundation)

How to Think in Java

Java is a strongly typed, object-oriented, platform-independent language designed for building reliable software at scale. You write source code once, compile it into bytecode, and execute it on any machine with a JVM. This is why Java is used in enterprise systems, backend APIs, Android, fintech platforms, and high-traffic web applications.

1. **Compile Once, Run Anywhere**

   Java source code is compiled to bytecode. The JVM executes bytecode, so your compiled program runs across operating systems without rewriting platform-specific code.

2. **OOP by Default**

   Java organizes code through classes and objects. Encapsulation, inheritance, polymorphism, and abstraction are first-class patterns, not optional style choices.

3. **Safety and Maintainability**

   Static typing catches many errors early. Automatic garbage collection removes most manual memory management bugs and improves long-term maintainability.

4. **Production Ecosystem**

   Tools like Maven, JUnit, and Spring Boot make Java practical for large teams: dependency management, testing, structure, and deployment are all standardized.

Execution Model (Source → Running Program)

`.java` source file → `javac` compiler → `.class` bytecode → JVM loads classes → JIT optimizes and executes

```
javac Main.java        # compiles to Main.class
java Main              # JVM starts at public static void main(Stri
ng[] args)
```

1. Basics, Memory, & Control Flow

Execution: `.java` (Source) ➔ `javac` (Compiler) ➔ `.class` (Bytecode) ➔ `JVM` (Execution)
Memory: * **Stack:** Stores method frames, local primitive values, and object references. Cleared when methods return.

- **Heap:** Stores actual Objects and Arrays. Handled by the Garbage Collector (GC).

Your First Java Program (Hello World)

Every Java application needs an entry point. The JVM looks for the `main` method to start running your code.

```
public class HelloWorld {
    // The main method is the exact starting point of your program
    public static void main(String[] args) {
        System.out.println("Hello, World!"); // Prints text to the console
    }
}
```

Breaking it down:

- `public`: Accessible from anywhere (the JVM needs to be able to see it).
- `class`: Everything in Java must live inside a class. The filename must match the public class name (`HelloWorld.java`).
- `static`: Belongs to the class itself. The JVM can run this method without needing to create an object of `HelloWorld` first.
- `void`: This method does not return any value when it finishes.
- `main`: The specific method name the JVM searches for.
- `String[] args`: Allows you to pass command-line arguments when starting the app.

Mental Model: Stack vs Heap

Area	Stores	Lifetime	Typical issues
Stack	Method frames, local primitive values, references	Until method returns	Deep recursion can cause stack overflow

Area	Stores	Lifetime	Typical issues
Heap	Objects and arrays	Until unreachable and GC collects	Too many live objects can increase memory pressure

Compilation Errors vs Runtime Errors

Type	When it happens	Examples	Fix strategy
Compile-time	Before app starts	Type mismatch, missing semicolon, unknown symbol	Fix syntax/type issues, then recompile
Runtime	While app is running	NullPointerException, ArrayIndexOutOfBoundsException	Validate input, add guards, handle exceptions

Practical rule: Learn Java in this order: syntax and control flow, methods, arrays, OOP, collections, exceptions, streams, ecosystem tools. This sequence matches how real backend apps are built.

Module 00A — Java Basics (Pre-OOP) — Merged Notes

Merged from `java-basics-notes.html`: the complete pre-OOP basics block is included here so you can study theory and coding fundamentals in one place.

01 — How Java Works

JVM & Compilation

You write `.java`, the compiler creates `.class` bytecode, and the JVM runs it on any machine. This is the base of Java portability and consistency.

▼ ☕ Hello World Breakdown

```java
public class HelloWorld {
    public static void main(String[] args) {

        // public  -> visible to JVM
        // static  -> callable without object
        // void    -> no return value
        // main    -> entry point
        System.out.println("Hello, World!");
```

```
    }
}
```

02 — Data Types

Primitives vs Objects

Type	Size	Example	Note
int	32-bit	<code>int x = 5;</code>	Most common integer type
long	64-bit	<code>long x = 99L; </code>	Use <code>L</code> suffix
double	64-bit	<code>double x = 3.14; </code>	Default decimal type
float	32-bit	<code>float x = 3.0f; </code>	Use <code>f</code> suffix
boolean	—	<code>boolean b = true; </code>	true or false
char	16-bit	<code>char c = 'A'; </code>	Single character
String	Object	<code>String s = "Ali"; </code>	Reference type

▼ ☕ Type Casting

```
int a = 5;
double b = a;           // implicit widening
int c = (int) 3.99;     // explicit narrowing -> 3

String num = String.valueOf(42);
int x = Integer.parseInt("42");
```

03 — String Methods

`String` is not primitive. It exposes methods for searching, slicing, formatting, and transformation.

▼ ☕ String Methods Reference

```
String s = "Hello World";

s.length();
s.toUpperCase();
```

```
s.toLowerCase();
s.charAt(0);
s.substring(0, 5);
s.contains("World");
s.replace("World", "Java");
s.trim();
s.isEmpty();
s.equals("Hello World");
s.split(" ");
s.indexOf("W");

String name = "Ali";
String msg  = "Hello, " + name + "!";
String msg2 = String.format("Hello, %s! Age: %d", name, 20);
```

String comparison rule: Use `.equals()` for text equality. Do not use `==` for String content checks.

04 — User Input

Scanner Class

▼ ☕ Reading Input

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

String name = sc.nextLine();
int age     = sc.nextInt();
double gpa  = sc.nextDouble();

int num = sc.nextInt();
sc.nextLine();           // clear leftover newline
String line = sc.nextLine();

sc.close();
```

Page 2 — Control Flow, Arrays, Methods

05 — Control Flow

If / Else / Switch

▼ ☕ Conditionals

```java
int score = 85;

if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else {
    System.out.println("C or below");
}

String result = (score >= 60) ? "Pass" : "Fail";

String dayType = switch ("MON") {
    case "SAT", "SUN" -> "Weekend";
    default -> "Weekday";
};
```

06 — Loops

For / While / Do-While

▼ ☕ All Loop Types

```java
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

int[] nums = {10, 20, 30};
for (int n : nums) {
    System.out.println(n);
}

int count = 0;
while (count < 3) {
    System.out.println(count++);
}

do {
    System.out.println("runs once minimum");
} while (false);
```

```java
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    if (i % 2 == 0) continue;
    System.out.println(i);
}
```

07 — Arrays

▼ ☕ Arrays

```java
int[] nums = {10, 20, 30, 40, 50};
int[] empty = new int[5];

nums[0];
nums[nums.length - 1];
nums.length;
nums[2] = 99;

for (int i = 0; i < nums.length; i++) {
    System.out.print(nums[i] + " ");
}

int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
matrix[1][2];

import java.util.Arrays;
Arrays.sort(nums);
Arrays.toString(nums);
Arrays.fill(empty, 7);
```

08 — Methods (Functions)

▼ ☕ Method Syntax & Patterns

```java
public static int add(int a, int b) {
    return a + b;
}

public static void greet(String name) {
    System.out.println("Hello, " + name);
```

```java
}

public static String getGrade(int score) {
    if (score >= 90) return "A";
    if (score >= 80) return "B";
    return "C";
}

int sum = add(5, 3);
greet("Ali");
String g = getGrade(85);

public static void tryChange(int x) { x = 999; }
int n = 5;
tryChange(n);
System.out.println(n);
```

Scope rule: Variables declared inside a method or loop only exist inside that block.

09 — Operators Quick Reference

▼ ☕ Operators

```java
// Arithmetic
+   // addition
-   // subtraction
*   // multiply
/   // divide
%   // remainder
++  // increment
--  // decrement

10 / 3 = 3
10 % 3 = 1

// Comparison (not for Strings)
==  // equal
!=  // not equal
>   // greater than
<   // less than
>=  // greater or equal
<=  // less or equal

"Ali".equals("Ali") // true
```

```
// Logical
&&  // AND
||  // OR
!   // NOT

x += 5;
x -= 2;
x *= 3;
x /= 2;
```

10 — Math Class

java.lang.Math

▼ ☕ Math Methods

```
Math.max(5, 9);
Math.min(5, 9);
Math.abs(-7);
Math.pow(2, 10);
Math.sqrt(16);
Math.round(3.7);
Math.floor(3.9);
Math.ceil(3.1);
Math.random();
int rand = (int)(Math.random() * 100);
Math.PI;
```

Quick Rules to Remember (Common Gotchas)

- **Integer division:** `10 / 3 = 3`. Cast to get decimals: `(double) 10 / 3`.
- **String comparison:** Use `.equals()` instead of `==` for text value checks.
- **Scanner newline issue:** After `nextInt()`, call `nextLine()` before reading a String line.
- **Fixed-size arrays:** Arrays cannot resize. Use `ArrayList` when size must grow dynamically.

📚 Table of Contents

- 03 Interfaces & Abstract Classes
- 04 ArrayList & HashMap
- 05 HashSet & Generics
- 06 Try / Catch / Finally
- 07 Custom Exceptions
- 08 Lambdas & Stream API
- 09 Optional & Records
- 10 Maven & JSON & Jackson

🏗️ Milestone 01 — Object-Oriented Programming

Java Basics —

Java runs on the **JVM (Java Virtual Machine)** — compile once, run anywhere. The JVM handles memory automatically via the Garbage Collector. No pointers, no `delete`, no header files. Every line of code must live inside a class.

🏗️ **Everything is a Class**
No global functions exist. Every method and variable lives inside a class. Filename MUST match class name exactly: `HelloWorld.java`.

♻️ **Automatic Memory**
Garbage Collector frees memory for you. No `delete` keyword. No memory leaks from forgetting to free objects.

📦 **String is an Object**
`String` is not a primitive. It's a full object with built-in methods: `.length()`, `.toUpperCase()`, `.contains()`, `.replace()`.

▼ ☕ Data Types & Your First Program

```
public class Main {
    public static void main(String[] args) {

        // ── Primitive types — stored directly in memory ──────
```

```
──────────
        int     age       = 20;
        double  gpa       = 3.85;
        boolean isStudent = true;
        char    grade     = 'A';
        long    bigNum    = 9999999999L;   // needs 'L' suffix
        float   temp      = 36.6f;         // needs 'f' suffix

        // ── Reference type — String is a full Object ──────────
──────────
        String name = "Ali Ahmed";

        // String has built-in methods
        name.length();                      // 9
        name.toUpperCase();                // "ALI AHMED"
        name.contains("Ali");              // true
        name.replace("Ali", "Sara");       // "Sara Ahmed"
        name.substring(0, 3);             // "Ali"

        // ── User Input ──────────────────────────────────────────
──────────
        Scanner sc = new Scanner(System.in);  // import java.uti
l.Scanner
        String  input  = sc.nextLine();        // reads full lin
e
        int     number = sc.nextInt();         // reads integer
        double  dec    = sc.nextDouble();      // reads decimal
        sc.close();
    }
}
```

Inheritance — `extends`

A child class inherits all non-private fields and methods from its parent. Models the **IS-A relationship**: a `Dog` IS-A `Animal`. Java supports single inheritance only — a class can extend only one parent.

⚠️ **super() Rule:** The child constructor MUST call `super()` as its very first line. It initializes the parent's fields. Java enforces this at compile time — forget it and your code won't build.

▼ ☕ Inheritance Chain

```java
// PARENT CLASS
public class Animal {
    protected String name;    // protected = accessible by child
classes
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    public void eat()   { System.out.println(name + " is eatin
g"); }
    public void sleep() { System.out.println(name + " is sleepin
g"); }
}

// CHILD CLASS — gets everything Animal has, plus its own stuff
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age);    // MUST be first — calls Animal's c
onstructor
        this.breed = breed;
    }

    public void bark() { System.out.println(name + ": Woof!"); }
}

// USAGE
Dog dog = new Dog("Bruno", 3, "Labrador");
dog.eat();         // ✅ inherited from Animal — Bruno is eatin
g
dog.sleep();       // ✅ inherited from Animal
dog.bark();        // ✅ Dog's own method — Bruno: Woof!

// super keyword — two jobs:
super(name, age);  // 1. Call parent constructor
super.eat();        // 2. Call parent method from inside child
```

Method Overriding (@Override) — Runtime Polymorphism

A child class **redefines** a method from the parent. Java automatically calls the child's version at runtime — even through a parent-type reference. This is the heart of polymorphism.

▼ ☕ Polymorphism in Action

```
class Animal {
    public void makeSound() { System.out.println("Generic soun
d"); }
}

class Dog extends Animal {
    @Override   // tells compiler: intentionally replacing paren
t's method
    public void makeSound() { System.out.println("Woof!"); }
}

class Cat extends Animal {
    @Override
    public void makeSound() { System.out.println("Meow!"); }
}

// THE POWER — parent reference holds child objects
Animal a1 = new Dog();    // Animal reference → Dog object
Animal a2 = new Cat();

a1.makeSound();           // Woof!  — runs Dog's version at runt
ime
a2.makeSound();           // Meow!  — runs Cat's version at runt
ime

// Array of parents holding different children
Animal[] zoo = { new Dog(), new Cat(), new Dog() };
for (Animal a : zoo) a.makeSound(); // Woof! Meow! Woof!

// instanceof — defensive check before casting
if (a1 instanceof Dog) {
    ((Dog) a1).bark();      // safe cast — we confirmed the type
first
}
```

Method Overloading — Compile-Time Polymorphism

▼ ☕ Overloading Example

```java
// Same name, different parameters — Java picks the right one at compile time
public class Calculator {
    public int    add(int a, int b)         { return a + b; }
    public double add(double a, double b)   { return a + b; }
    public int    add(int a, int b, int c) { return a + b + c; }
    public String add(String a, String b)  { return a + b; }
}

calc.add(2, 3);         // int version    → 5
calc.add(2.5, 3.5);   // double version → 6.0
calc.add(1, 2, 3);    // 3-param version → 6
calc.add("Hi", "!"); // String version → "Hi!"
```

Interfaces — Contracts

An interface defines **what** a class must do, not how. Any class that `implements` it MUST provide every method. A class can implement **multiple** interfaces (unlike inheritance).

> 🍃 **Spring Boot Connection:** Spring's entire security is built on interfaces like `UserDetailsService`. You implement it → Spring calls your code automatically during login. This is how frameworks plug into your code without seeing it in advance.

▼ ☕ Interface + Multiple Implements

```java
// THE CONTRACT — no body on methods
public interface Drawable {
    void draw();
    void resize(int factor);
}

public interface Saveable {
    void save();
}

// Implements MULTIPLE interfaces — classes can only extend ONE parent
public class Circle implements Drawable, Saveable {

    @Override
```

```
    public void draw()              { System.out.println("Drawing
circle ⭕"); }

    @Override
    public void resize(int factor) { System.out.println("Resize
x" + factor); }

    @Override
    public void save()              { System.out.println("Savin
g..."); }
}

// Polymorphism through interfaces
Drawable d = new Circle();   // Interface type, Object instance
d.draw();                       // Drawing circle ⭕

// Spring Boot does this — interface reference, your implementat
ion:
// UserDetailsService service = new MyUserService();
// service.loadUserByUsername("ali"); // calls YOUR code
```

Abstract Classes — The Middle Ground

Between a regular class and an interface. Has abstract methods (no body — child MUST override) AND concrete methods (with body — inherited as-is). **Cannot be instantiated directly.**

Feature	Interface	Abstract Class
Constructor	❌ No	✅ Yes
Instance variables	❌ No	✅ Yes
Multiple inheritance	✅ Many interfaces	❌ One class only
Use when	Defining a **capability** (CAN-DO)	Defining a **base type** (IS-A with shared logic)

▼ ☕ Abstract Class Example

```
public abstract class Shape {
    protected String color;

    public Shape(String color) { this.color = color; }

    // Abstract — NO body. Every subclass MUST implement this.
```

```
    public abstract double calculateArea();

    // Concrete — has body, all subclasses inherit this as-is.
    public void displayInfo() {
        System.out.println("Color: " + color + " | Area: " + cal
culateArea());
    }
}

class Circle extends Shape {
    double radius;
    public Circle(String c, double r) { super(c); this.radius =
r; }

    @Override
    public double calculateArea() { return Math.PI * radius * ra
dius; }
}

Shape s = new Circle("Red", 5);
s.displayInfo();   // Color: Red | Area: 78.53...
// new Shape() → ❌ COMPILE ERROR — cannot instantiate abstract
class
```

📦 Milestone 02 — Collections & Generics

✅ Milestone Complete

Arrays are fixed-size with no built-in operations. Java's Collections Framework provides dynamic, feature-rich data structures used in every real application and Spring Boot project.

Collection	Duplicates	Ordered	Access By	Best For
`ArrayList<T>`	✅ Allowed	✅ Yes	Index `get(0)`	Ordered, dynamic lists
`HashMap<K,V>`	Keys: ❌ Values: ✅	❌ No	Key `get("id")`	Fast key-value lookups
`HashSet<T>`	❌ Never	❌ No	`contains()` only	Unique value collections

ArrayList — Dynamic Arrays

▼ ☕ ArrayList Complete Reference

```
import java.util.ArrayList;

ArrayList<String> list = new ArrayList<>();

// Adding
list.add("Ali");              // add to end
list.add(0, "Sara");          // insert at index 0

// Reading
list.get(0);                   // "Sara" — access by index
list.size();                   // number of elements
list.contains("Ali");         // true/false
list.isEmpty();                // true if empty
list.indexOf("Ali");          // index of first occurrence

// Updating
list.set(0, "Zara");          // replace index 0 with "Zara"

// Removing
list.remove("Ali");           // remove by value
list.remove(0);               // remove by index
list.clear();                 // remove everything

// Looping
for (String item : list) { System.out.println(item); }  // enhan
ced for-each
list.forEach(item -> System.out.println(item));          // lamb
da way
```

HashMap — Key-Value Pairs

💡 **JSON IS a HashMap.** `{"name":"Ali","age":20}` is String→Value pairs. Every REST API on the internet uses this format. Master HashMap, master JSON.

▼ ☕ HashMap Complete Reference

```
import java.util.HashMap;

HashMap<String, Integer> grades = new HashMap<>();
```

```
// Adding / Updating
grades.put("Ali", 95);          // add key-value pair
grades.put("Ali", 98);          // OVERWRITES — duplicate keys n
ot allowed

// Reading
grades.get("Ali");              // 98   (null if key missing)
grades.getOrDefault("X", 0);  // 0    (safe — returns default if
missing)
grades.containsKey("Ali");     // true
grades.containsValue(95);      // true
grades.size();                  // number of entries

// Removing
grades.remove("Ali");          // removes that key entirely

// Looping — entrySet gives key AND value together
for (HashMap.Entry<String, Integer> entry : grades.entrySet()) {
    System.out.println(entry.getKey() + " → " + entry.getValue
());
}

// HashMap with Object values — real Spring Boot style
HashMap<String, Student> database = new HashMap<>();
database.put("S001", new Student("Ali", 20));
Student found = database.get("S001");    // instant lookup by ID
```

HashSet — Uniqueness Guaranteed

▼ ☕ HashSet & Duplicate Detection

```
HashSet<String> enrolled = new HashSet<>();

enrolled.add("Ali");
enrolled.add("Sara");
enrolled.add("Ali");         // SILENTLY IGNORED — already exist
s
enrolled.size();             // 2, not 3
enrolled.contains("Ali");  // true — extremely fast O(1) lookup
enrolled.remove("Sara");

// Classic interview pattern — find duplicates in O(n)
```

```
int[] nums = {1, 2, 3, 2, 4, 3};
HashSet<Integer> seen = new HashSet<>();
for (int n : nums) {
    if (!seen.add(n))           // add() returns false if alread
y present
        System.out.println("Duplicate: " + n);
}
// Output: Duplicate: 2   Duplicate: 3
```

Generics <T> — Type Safety at Compile Time

Generics let you write **one class or method that works safely with any type**. Errors are caught at compile time instead of crashing at runtime. Spring Boot's `JpaRepository` uses generics heavily.

▼ ☕ Generic Class & Methods

```
// WITHOUT generics — dangerous, crashes at runtime
ArrayList rawList = new ArrayList();
rawList.add("Hello");
rawList.add(42);              // Java allows this — chaos!
String s = (String) rawList.get(1); // CRASH — 42 is not a Strin
g

// WITH generics — safe, compile-time protection
ArrayList<String> safeList = new ArrayList<>();
safeList.add(42);             // ❌ COMPILE ERROR — caught befo
re running ✅

// YOUR OWN generic class — T is a placeholder for "any type"
public class Box<T> {
    private T content;
    public Box(T content)  { this.content = content; }
    public T getContent()  { return content; }
}

Box<String>  b1 = new Box<>("Hello");  // T becomes String
Box<Integer> b2 = new Box<>(42);        // T becomes Integer
Box<Student> b3 = new Box<>(new Student(...)); // T becomes Stud
ent
String val = b1.getContent();             // No cast needed — ty
pe is known ✅

// Spring Boot uses this pattern for database access:
```

```
// JpaRepository<Student, Long> — first = entity type, second =
ID type
// save(), findById(), findAll() all work automatically for YOUR
entity
```

🛡️ Milestone 03 — Exception Handling

✅ Milestone Complete

Without exception handling, one bad user input crashes your entire server for every user. With it — catch the problem, respond cleanly, keep the server running.

Try / Catch / Finally

🔒 **try { }**
Wrap risky code here. If any line throws, execution immediately jumps to the matching catch block.

🛡️ **catch (Ex e) { }**
Handle the exception. `e.getMessage()` describes what went wrong. Multiple catch blocks — most specific FIRST.

🧹 **finally { }**
Runs NO MATTER WHAT — crash or success. Close files, DB connections, release resources here.

▼ ☕ Multiple Catch Blocks

```
try {
    int   idx  = Integer.parseInt(sc.nextLine()); // NumberForma
tException
    int[] nums = {1, 2, 3};
    System.out.println(nums[idx]);              // ArrayIndexOu
tOfBoundsException
    int   res  = nums[0] / 0;                   // ArithmeticEx
ception

} catch (NumberFormatException e) {
    System.out.println("Not a number! " + e.getMessage());

} catch (ArrayIndexOutOfBoundsException e) {
```

```
    System.out.println("Index 0–2 only!");

} catch (Exception e) {
    System.out.println("Unexpected: " + e.getMessage());  // ALW
AYS LAST

} finally {
    sc.close();   // runs whether exception happened or not
}

System.out.println("Server keeps running!"); // executes after c
atch
```

Checked vs Unchecked Exceptions

Type	Cause	Java Forces Handle?	Common Examples
Unchecked (RuntimeException)	Your code logic is wrong	❌ Optional	`NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`
Checked (Exception)	External world: missing file, network down, DB gone	✅ Compile error if ignored	`IOException`, `FileNotFoundException`, `SQLException`

▼ ☕ Checked Exception & throws Keyword

```
// Checked — Java FORCES you to handle this (file might not exis
t)
try {
    FileReader file = new FileReader("data.txt");  // FileNotFou
ndException
} catch (FileNotFoundException e) {
    System.out.println("File missing: " + e.getMessage());
}

// throws — pass responsibility to the caller instead of handlin
g here
public static void readFile(String path) throws IOException {
    FileReader f = new FileReader(path);  // no try-catch needed
here
}

// Caller MUST handle it
```

```
try { readFile("data.txt"); }
catch (IOException e) { System.out.println("Error: " + e.getMess
age()); }
```

Custom Exceptions — Professional Standard

Create meaningful, domain-specific exceptions for your application. This is how every production Spring Boot app works.

▼ ☕ Custom Exception Classes

```
// Extend RuntimeException = Unchecked (Spring Boot convention)
public class UserNotFoundException extends RuntimeException {
    private String userId;

    public UserNotFoundException(String userId) {
        super("User not found with ID: " + userId);
        this.userId = userId;
    }

    public String getUserId() { return userId; }
}

public class InsufficientBalanceException extends RuntimeExcepti
on {
    private double amount;

    public InsufficientBalanceException(double amount) {
        super("Insufficient balance. Tried to withdraw: $" + amo
unt);
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

// Throwing custom exceptions in your service methods
public Student findStudent(String id) {
    Student s = database.get(id);
    if (s == null) throw new UserNotFoundException(id);  // stop
s execution
    return s;
}
```

```
// Spring Boot Global Handler — catches ALL exceptions across en
tire app
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handle(UserNotFoundException
e) {
        return ResponseEntity.status(404).body(e.getMessage());
// clean JSON error
    }
}
```

⚡ Milestone 04 — Modern Java

✅ Milestone Complete

Lambdas — Anonymous Functions

Lambdas collapse verbose anonymous class syntax into clean one-liners. Spring Boot 3 code is written almost entirely with lambdas.

▼ ☕ Lambda Syntax — All Forms

```
// Syntax:  (parameters) -> expression   or   (parameters) -> {
body; }

() -> System.out.println("No params")          // no parameters
name -> System.out.println("Hi " + name)       // one param — pa
rens optional
(a, b) -> a + b                                  // multiple pa
rams
(a, b) -> { int sum = a+b; return sum; }        // multi-line ne
eds braces + return

// Lambdas with collections
List<String> names = new ArrayList<>(List.of("Ali", "Sara", "Bo
b"));

names.forEach(n -> System.out.println(n));      // lambda
names.forEach(System.out::println);             // method refere
```

```
nce — shortest form

names.sort((a, b) -> a.compareTo(b));          // alphabetical
names.sort((a, b) -> a.length() - b.length()); // by length
names.sort((a, b) -> b.compareTo(a));          // reverse alphab
etical
```

Stream API — Pipeline Data Processing

Streams process collections through a **chain of operations** — no temp variables, no messy loops.

🔄 List / Collection → `.stream()` → `.filter()` → `.map()` → `.sorted()` → `.toList()` / `.count()` / etc.

▼ ☕ Stream Operations — Complete Reference

```
List<Integer> nums = List.of(1,2,3,4,5,6,7,8,9,10);

// ── INTERMEDIATE OPERATIONS (return a new stream) ──────────────
─
nums.stream().filter(n -> n % 2 == 0)     // keep evens: [2,4,6,
8,10]
nums.stream().map(n -> n * 2)             // double each: [2,4,
6,...,20]
nums.stream().sorted()                     // ascending order
nums.stream().distinct()                   // remove duplicates
nums.stream().limit(5)                    // first 5 only

// ── TERMINAL OPERATIONS (produce a final result) ──────────────
─
nums.stream().toList()                     // collect to List
nums.stream().count()                      // how many elements
nums.stream().reduce(0, Integer::sum)      // sum all = 55
nums.stream().anyMatch(n -> n < 0)        // false
nums.stream().allMatch(n -> n > 0)        // true
nums.stream().findFirst()                  // Optional<Integer>
nums.stream().mapToInt(n -> n).sum()      // 55
nums.stream().mapToInt(n -> n).average()  // OptionalDouble

// ── CHAINING — the real power ──────────────────────────────────
──
```

```java
List<String> result = nums.stream()
    .filter(n -> n % 2 == 0)      // [2,4,6,8,10]
    .map(n -> "num_" + n)          // ["num_2","num_4",...]
    .sorted()
    .toList();

// ── WITH OBJECTS — Spring Boot daily pattern ──────────────────
List<Student> topCS = students.stream()
    .filter(s -> s.getMajor().equals("CS"))   // only CS students
    .filter(s -> s.getGpa() > 3.5)            // high GPA only
    .toList();

double avgGpa = students.stream()
    .mapToDouble(s -> s.getGpa())
    .average()
    .orElse(0.0);
```

Optional — Null Safety

`Optional<T>` is a container that explicitly represents *"this might or might not have a value."* Spring's repository methods return Optional — you must handle the empty case.

▼ ☕ Optional — Complete Reference

```java
// Creating Optionals
Optional<String> present = Optional.of("Ali");          // definitely has value
Optional<String> empty   = Optional.empty();             // definitely empty
Optional<String> maybe   = Optional.ofNullable(null);   // might be null — safe

// Reading safely
present.isPresent();                          // true
present.isEmpty();                            // false (Java 11+)
present.get();                                // "Ali" — ONLY safe if isPresent() first!
empty.orElse("Default");                     // "Default" — fallback value
empty.orElseGet(() -> computeDefault());     // lazy computation of default
```

```
empty.orElseThrow(() -> new RuntimeException("Not found")); // t
hrow if empty
present.ifPresent(v -> System.out.println(v)); // run only if va
lue exists

// Stream + Optional — THE Spring Boot pattern you'll write ever
y day
Student student = students.stream()
    .filter(s -> s.getName().equals("Ali"))
    .findFirst()                                  // returns Option
al<Student>
    .orElseThrow(() -> new UserNotFoundException("Ali"));
```

Records — Instant Data Classes (Java 16+)

A `record` auto-generates constructor, getters, `toString()`, `equals()`, and `hashCode()`. Immutable by design. Perfect for DTOs in Spring Boot.

▼ ☕ Records Example

```
// OLD WAY — 35+ lines
public class StudentOld {
    private final String name; private final int age; private fi
nal double gpa;
    public StudentOld(String name, int age, double gpa) { ... }
    public String getName() { return name; }  // + getAge(), get
Gpa(), toString()...
}

// RECORD WAY — 1 line does everything above
public record Student(String name, int age, double gpa) {}

// Auto-generated: constructor, accessors (no "get" prefix), toS
tring, equals
Student s = new Student("Ali", 20, 3.9);
s.name();          // "Ali"
s.age();           // 20
s.gpa();           // 3.9
System.out.println(s); // Student[name=Ali, age=20, gpa=3.9]

// Records + Streams — clean modern Java
record Product(String name, double price) {}
List<String> expensive = products.stream()
    .filter(p -> p.price() > 500)
```

```
    .map(p -> p.name())
    .toList();

// Records are IMMUTABLE — no setters. Ideal for Spring Boot @Re
questBody DTOs.
```

🌐 Milestone 05 — The Ecosystem: Maven & JSON

✅ Milestone Complete

Maven — Build Tool & Dependency Manager

Maven manages dependencies, builds, and packaging. Instead of manually downloading 50+ `.jar` files, you declare what you need in `pom.xml` and Maven downloads everything automatically.

📋 **pom.xml**
Project Object Model. The project's brain. Declares who you are, what you need, and how to build.

🌐 **Maven Central**
Public repository of millions of Java libraries. Maven downloads from here automatically when you add a dependency.

📦 **Starters**
Spring Boot starters like `spring-boot-starter-web` bundle 50+ libraries into one dependency line.

▼ 📄 Spring Boot pom.xml — Annotated

```
<!-- Spring Boot Parent handles dependency versions for you -->
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
</parent>

<groupId>com.yourname</groupId>
```

```
<artifactId>student-api</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
    <java.version>21</java.version>
</properties>

<dependencies>
    <!-- Web — REST APIs, embedded Tomcat, JSON (50+ libs in 1 l
ine) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Database — JPA + Hibernate ORM -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- MySQL driver — runtime scope = not needed to compile, o
nly to run -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Security — login, JWT, authentication -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
</dependencies>
```

Maven Commands — Daily Usage

```
mvn clean install        # clean → compile → test → package  (use t
his most)
mvn compile              # compile only — check for errors
mvn test                 # run all unit tests
```

```
mvn package              # create runnable .jar in /target folder
mvn spring-boot:run      # start your Spring Boot app locally
```

Maven Standard Project Structure

```
my-app/
├── pom.xml                                  # Maven brain — dependenci
es declared here
├── src/
│   ├── main/
│   │   ├── java/com/yourname/app/
│   │   │   ├── MainApplication.java         # @SpringBootApplication e
ntry point
│   │   │   ├── StudentController.java       # HTTP layer
│   │   │   ├── StudentService.java          # Business logic
│   │   │   └── StudentRepository.java       # Database layer
│   │   └── resources/
│   │       └── application.properties       # DB URL, port, config
│   └── test/
│       └── java/                            # Unit tests mirror main s
tructure
└── target/                                  # Compiled output, generat
ed .jar
```

JSON — The Language of the Internet

Every web API communicates in JSON. Spring Boot converts between Java objects and JSON automatically via the **Jackson** library.

▼ 📄 JSON Syntax Reference

```
{
    "name":     "Ali Ahmed",
    "age":      20,
    "gpa":      3.9,
    "enrolled": true,
    "nickname": null,
    "courses":  ["Math", "CS101"],
    "address":  {
        "city":    "Lahore",
        "country": "Pakistan"
    }
}
```

```
// Rules: Keys ALWAYS in double quotes. No trailing commas. No c
omments.
```

Jackson — Java ↔ JSON

▼ ☕ Serialization & Deserialization

```
// Jackson's ObjectMapper does the conversion
ObjectMapper mapper = new ObjectMapper();

// SERIALIZATION — Java Object → JSON String
StudentDTO student = new StudentDTO("Ali", 20, 3.9);
String json = mapper.writeValueAsString(student);
// → {"name":"Ali","age":20,"gpa":3.9}

// DESERIALIZATION — JSON String → Java Object
String incoming = """{"name":"Sara","age":21,"gpa":3.7}""";
StudentDTO received = mapper.readValue(incoming, StudentDTO.clas
s);

// ── IN SPRING BOOT — Jackson is completely invisible ──────────
@GetMapping("/students/{id}")
public StudentDTO getStudent(@PathVariable Long id) {
    return new StudentDTO("Ali", 20, 3.9);
    // ↑ Spring + Jackson converts this to JSON automatically
}

@PostMapping("/students")
public String create(@RequestBody StudentDTO student) {
    // ↑ Jackson converts incoming JSON to StudentDTO automatica
lly
    return "Created: " + student.getName();
}

// For Jackson to work, your DTO class needs:
// 1. No-argument constructor  2. Getters  3. Setters
```

🚀 What's Next — Spring Boot

🎯 Every milestone maps directly to a layer of a real Spring Boot application. Nothing was learned without purpose.

```
Spring Boot REST API — Student Management System
│
├── StudentController          ← HTTP layer (GET, POST, PUT, DELETE)
│   uses: OOP (M1), Annotations, Records as DTOs (M4)
│
├── StudentService             ← Business logic
│   uses: Collections (M2), Streams + Lambdas (M4), Optional (M4)
│
├── StudentRepository          ← Database access layer
│   extends JpaRepository<Student, Long>
│   uses: Generics (M2), Interfaces (M1)
│
├── Student                    ← Database entity
│   uses: Records or OOP (M1, M4), Encapsulation
│
├── StudentDTO                 ← JSON in / JSON out
│   uses: Jackson (M5), Records (M4)
│
├── GlobalExceptionHandler     ← Catches all errors app-wide
│   uses: Custom Exceptions (M3)
│
└── pom.xml                    ← Pulls in all Spring Boot libraries
    uses: Maven (M5)
```

🏆 **You are ready.** All 5 milestones complete. OOP · Collections · Exceptions · Modern Java · Maven · JSON — the foundations every Spring Boot developer uses daily.

"The best way to learn Java is to build something real with it."

5 Milestones · Java Core Complete · Spring Boot Next →